Ayush Giri

Oracle —— Senior engineer

scaler —— 2022

400+ lectures

DSA, Competitive, LLD

Coding ninjas, Newton school.

DSA

Intermediate

1 months

→ Power of observation + TC and SC

→ Arrays —— Manipulations

→ Lab —— TC, SC debugging

→ Prefix sum & carry forward

→ sliding window & contribution

→ Memory management & sorting

→ Bit manipulation

→ 2D matrices

→ strings

→ Lab – 2D matrices, strings

Advanced

3-4 month

Problems

4 problems

200-225 problems

Contest

Classes

9:05 — start

10:20 ——— Part 1

Break —— 10 minutes

10:30 ——— 11:15

After: Doubts [11:30]

Agenda

- Power of observation
  - count of factors
  - sum of n natural no
  - GP & sum of GP.
  - Iterations
  - Comparing algo
  - TC. — Big O

Qu Given a no n. find no of factors.

$n \% i == 0$

factor of n

n=10 —— 1 2 3 4 5 6 7 8 9 10

10%1==0

Ans = 4

n=24 —— 1 2 3 4 6 8 12 24 Ans = 8

n=12 —— 1 2 3 4 5 6 7 8 9 10 11 12

```
int countfactors(int n) {
    int cnt = 0;
    for(i=1; i<=n; i++) {
        if(n % i == 0) {
            cnt ++;
        }
    }
    return cnt;
}
```

itr = n.

Execution Time : Depends on system & environment

Game:

Assumption

10^8 itr are executed in 1 sec.

```
int countfactors(int n) {
    int cnt = 0;
    for(i=1; i<=n; i++) {
        if(n % i == 0) {
            cnt++;
        }
    }
    return cnt;
}
```

itr = n.

n	# itr	Execution Time
10^8	10^8	1 sec.
10^9	10^9	10^8 itr — 1 sec 1 itr — $\frac{1}{10^8}$ sec 10^9 itr — $\frac{10^9}{10^8} = 10$ sec
10^{18}	10^{18}	10^{18} itr — $\frac{10^{18}}{10^8} = \boxed{10^{10} \text{ sec}}$

317 years

you → children →

grandchildren → 4th/5th

Approach2

$i * j = n$. { i & j are factors of n }

$j = n/i$ { i & $\frac{n}{i}$ " " " " }

$n = 24$

i	n/i
1	$24/1 = 24$
2	$24/2 = 12$ — P1
3	$24/3 = 8$
4	$24/4 = 6$
6	$24/6 = 4$
8	$24/8 = 3$ — P2
12	$24/12 = 2$
24	$24/24 = 1$

→ P1 has all the answers

$i <= \frac{n}{i}$

$i^2 <= n$

$i <= \sqrt{n}$.

$i = 1$ —— $\sqrt{n}$.

$n = 100$

i	n/i
1	$100/1 = 100$
2	$100/2 = 50$
4	$100/4 = 25$
5	$100/5 = 20$
10	$100/10 = 10$
20	5
25	4
50	2
100	1

i & n/i are same

+
−
*
÷
} — constant

```
int countfactorsoptimised ( n ) {
    cnt = 0;
    for(i = 1 ; i <= sqrt(n) ; i++) {
        (or)
        i * i <= n.
        if (n % i == 0) {
            if ( i == n/i ) {
                cnt += 1;
            }
            else {
                cnt += 2;
            }
        }
    }
    return cnt;
}
```

$\log_2 n$

itr = $\sqrt{n}$.

n	# itr = $\sqrt{n}$	Execution Time
10^8		
10^9		
10^{18}	$\sqrt{10^{18}} = (10^{18})^{1/2} = 10^9$	10^8 itr — 1 sec 1 itr — $\frac{1}{10^8}$ sec 10^9 itr — $\frac{10^9}{10^8} = 10$ sec

317 years ⟶ 10 sec

Qu

10 —— not prime
11 —— prime
23 —— prime
2 —— prime
25 —— Not prime
27 —— " "
31 —— prime

Prime no

↑

2 factors
[1 & n]

Qu Given n. check if its prime or not ?

```
boolean isPrime(int n) {
    cnt = countfactorsoptimised(n);
    if(cnt==2) {
        return true;
    }
    return false;
}
```

Maths

$[3, 10]$ —— $3, 4, 5, 6, 7, 8, 9, 10$ —— 8 Ans

$10 - 3 + 1$ —— 8 Ans

inclusive.

$[a, b]$ —— $b - a + 1$ el

$(3, 10)$ —— $4, 5, 6, 7, 8, 9,$ —— 6 Ans

$10 - 3 - 1$ —— 6 Ans

exclusive

(a, b) —— $b - a - 1$ els

Qu $1 + 2 + 3 + 4 \cdots\cdots 100$

$$s = 1 + 2 + 3 + 4 \cdots\cdots 98 + 99 + 100$$

$$s = 100 + 99 + 98 + 97 \cdots\cdots 3 + 2 + 1$$

$+$ $+$

$$2s = 101 + 101 + 101 + 101 + \cdots\cdots 101 + 101 + 101$$

100 times

$$2s = 101 * 100$$

$$s = \frac{101 * \cancel{100}^{50}}{\cancel{2}} = \boxed{5050} \text{ Ans}$$

Generalisation

$$s = 1 + 2 + 3 + 4 \cdots\cdots (n-2) + (n-1) + n$$

$$s = n + (n-1) + (n-2) + (n-3) \cdots\cdots 3 + 2 + 1$$

$+$ $+$

$$2s = (n+1) + (n+1) + (n+1) + (n+1) \cdots\cdots (n+1) + (n+1) + (n+1)$$

n times

$$2s = (n+1) * n$$

$$\boxed{s = \frac{n(n+1)}{2}}$$

Geometric progression

5, 10, 20, 40, 80, 160, 320 . . . . . .

*2 *2 *2 *2 *2 *2

$a = 5$ first term

$r = 2$ common ratio

nth term of GP: $a * r^{n-1} = 5 * 2^{4} = 5 * 16 = 80$

5th term: 5 10 20 40 80

Sum of GP: $\dfrac{a \cdot (r^{n} - 1)}{r - 1}$ $r \neq 1$

Break: 10:33 – 10:40

```
for(i=1 ———————— n) {
        if (i==n) {
            break;
        }
}
```

n Times

n = 5 ⟶ 5 Time

i = 1 ✓
i = 2 ✓
i = 3 ✓
i = 4 ✓
i = 5 ✓

Big O ——

itr = n.

Big O — O(n)

```
for(i=0 ———————— 100) {
        - - - - - - - - -
}
```

i = 0
= 1
= 2
⋮
100

101 Times

Big O ——

itr — 101

O(1) Ans

```
for(i=1 ———————— n) {
        if (i%2==0) {
            ————
        }
}
```

n

```
for(i=1 ———————— m) {
        if (i%2==0) {
            ————
        }
}
```

m

n + m Ans

O (n + m)

Comparing 2 Algorithms

Story:

Ideal

Anuj	Nripen
Execution time	
15 sec	10 sec — Nripen winner
xp	M2 chip
M2 chip	
7 sec	10 sec — Anuj
C++	Python
	C++
7 sec	5 sec — Nripen
top of volcano	Mt. everest
Mt everest	
5 sec	5 sec — Equal

Idea2 : Big O complexity { 1. Comparison criteria
2. Worst case complexity of any algorithm }

— Calculate # of iterations

— Ignore lower order terms

— " constant co-efficients

Ex Yatan: # itr = $100 \log_2 n$

Big O = $O(\log_2 n)$

Jayanta # itr = $4n + 3n^2 + 60 \log_2 n$

Big O = $O(n^2)$

itr = $4n + 3n \log n + 1$

Big O = $O(n \log n)$

itr = $4n \log_2 n + 3n\sqrt{n} + 10^6$

$O(n\sqrt{n})$

$n = 100$

$\sqrt{n} = 10$

$\log_2 100 = 6.$ ____

$$\log_2 n < \sqrt{n} < n < n \log n < n\sqrt{n} < n^2 < n^3 \cdots < 2^n < n! < n^n$$

Why do we neglect lower terms?

Example

Bharath $\longrightarrow$ # itr $= n^2 + 10n$

Big O $= O(n^2)$

Lower order: $10 * n$

Higher order: n^2

input size	Itr $[n^2 + 10n]$	% of lower term contribution in total itr
$n = 10$	$10^2 + 10*10 = 200$	L.O $= 10*10 = 100$ % $= \frac{\cancel{100}}{\cancel{200}\ 2} * 100\% = 50\%$
$n = 100$	$(100)^2 + 10*100$ $= 10000 + 1000$ $= 11000$	L.O $= 10*100 = 1000$ % $= \frac{1\cancel{000}}{11\cancel{000}} * 100\% = \frac{100}{11}\% = 9.__\%$
$n = 10^4$	$(10^4)^2 + 10*10^4$ $10^8 + 10^5$	L.O $= 10*10^4 = 10^5$ % $= \frac{10^5}{10^8 + 10^5} * 100\% = 0.____\%$

Conclusion: As ip $\uparrow$, contribution of lower terms $\downarrow$ significantly.

Thankyou (:)

Mob : 8287457773